EXPERIMENT 7

Write a Python program to implement BFS

Register Number: **192511013**

192511013
192511013
192511013

Aim

To write a Python program to implement the Breadth First Search (BFS) algorithm, which traverses a graph level by level starting from a source node, and to use it to find the shortest path (in number of edges) between two nodes.

Algorithm

Step 1: Represent the graph as an adjacency list (a dictionary mapping each node to its list of neighbours).

Step 2: Create an empty queue and a visited set. Insert the start node into the queue and mark it as visited.

Step 3: Remove the node at the front of the queue and add it to the traversal order.

Step 4: Visit each neighbour of that node. If a neighbour has not been visited, mark it as visited and insert it at the rear of the queue.

Step 5: Repeat steps 3 and 4 until the queue becomes empty. The recorded order is the BFS traversal.

Step 6: For the shortest path, store the parent of every node when it is first discovered. When the goal node is removed from the queue, follow the parent links back to the start and reverse them.

Step 7: Print the traversal order and the shortest path.

192511013

Program

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    return order

def bfs_shortest_path(graph, start, goal):
    parent = {start: None}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for neighbour in graph[node]:
            if neighbour not in parent:
                parent[neighbour] = node
                queue.append(neighbour)
    return None

if __name__ == "__main__":
    print("Graph (adjacency list):")
    for node, adj in graph.items():
        print(f"  {node} -> {adj}")
    print("\nBFS traversal from A:", ' -> '.join(bfs(graph, 'A')))
    path = bfs_shortest_path(graph, 'A', 'F')
    print("Shortest path from A to F:", ' -> '.join(path))
    print("Number of edges in the path:", len(path) - 1)
```

192511013

Output

```
Graph (adjacency list):
  A -> ['B', 'C']
  B -> ['A', 'D', 'E']
  C -> ['A', 'F']
  D -> ['B']
  E -> ['B', 'F']
  F -> ['C', 'E']

BFS traversal from A: A -> B -> C -> D -> E -> F
Shortest path from A to F: A -> C -> F
Number of edges in the path: 2
```

192511013